PROJECT REPORT

UpLift Inc.

Project 7

A tower defence game with a telemetry measuring system and difficulty balancing dashboard for developers to ensure fair gameplay

Thomas Wilkin-Jones, Oliver Robson, Jake Phillips, Doruk Aksu, Dylan Davies, Bela Rothfuss Moore, Zak Salmon

Contents

0. AI Statement 2
1. Introduction 2
1.1. The Problem 2
1.2. Aims and Objectives 3
1.2.1. Core Components 3
1.2.2. User Stories 3
1.2.3. Sprint 1 Stories and Goals 4
1.2.4. Sprint 2 Stories and Goals 4
1.3. What We Built 6
1.3.0. Success 6
1.3.1. Sprint 1 Product 6
1.3.2. Sprint 2 Product 7
1.4. Outside the Scope 9
2. Design and Related Work 10
2.1. Early Prototyping 10
2.2. Architecture and Dataflow 10
2.2.1. Rationale 11
2.3. Similar Approaches 12
2.4. Scientific Advances 12
3. Implementation, Methods and Materials 12
3.1. Key Components 12
3.1.1. The Game 12
3.1.2. The Database 13
3.1.3. The Dashboard 14
3.2. Requirements and Implemented Features 15
3.2.0. Requirements 15
3.2.1. Sprint 1 Features 15
3.2.2. Sprint 2 Features 17
3.3. Tools 18
3.4. Data Analysis 18
3.5. Analysis Methods 18
4. Testing and Evaluation 18
4.1. Tests 18
4.1.1. Metrics 19
4.1.2. Results 19
4.2. Evaluation Approach 20
4.3. Evaluation Results 20

5. Management 21
5.1. Teamwork 21
5.2. Contributions and Roles 21
5.2.1. Sprint 1 21
5.2.2. Sprint 2 22
6. Risks and Discussion 23
6.1. Strategy 23
6.1.1. Pros 23
6.1.2. Cons 23
6.2. Project Failure Risk 24
6.2.1. Sprint 1 Risks 24
6.2.2. Sprint 2 Risks 24
6.3. Project Failure Prevention 24
6.3.1. Sprint 1 Risk Mitigation 24
6.3.2. Sprint 2 Risk Mitigation 25
7. Final Evaluation 26
7.1. Success 26
7.2. Trade-Offs 26
7.3. Scope Drift 26
7.4. Consistency and Teamwork 26
7.5. Next Steps 26
8. Appendices 27
8.1. Setup Instructions 27
8.2. API Documents, Endpoints and Schema 27
8.3. Test Plan and Test Coverage 27
9. Report Credits 27

0. AI Statement

This project complies with the AI minimal requirements. No AI has been used for code generation or code autocomplete. No AI has been used in the creation, planning or updating of any documents, databases, presentations, meeting minutes, slides, scripts, assets, menus, user interfaces or web pages.

1. Introduction

1.1. The Problem

The problem we have focused on is the task of making sure video games are fair and enjoyable for all players. This is not an easy thing for developers to understand and balance, as there is a lot of natural bias, since they

are the ones who have created the game, and therefore understand its inner workings and functionality better than the average player would. This often leads to gameplay being too challenging for a majority of players, and it is difficult for developers with less resources to find a fair difficulty level if they only have their own gameplay runs to analyse.

1.2. Aims and Objectives

1.2.1. Core Components

We were tasked with developing a simple single-player game which was capable of recording a number of event types during gameplay, and which had a series of adjustable parameters to tweak gameplay aspects and difficulty. This was designed to be flexible and easy to understand, acting as a guide for how the data collection and telemetry displays would work alongside a similar game.

Alongside this, we aimed to build a database to collect a series of events and related data while the game is being played. Every time an event occurs in the game, such as stage_start, stage_fail, stage_end, and enemy_defeated, the database should record an event, so the developer can analyse timestamps and trends to find common failure points in the game. These should all have relevant payloads which are sent from the game to the database, with appropriate parameters such as enemy_id for any enemy-related event types, so they can all be analysed effectively and in full detail.

This database would then feed data into a telemetry analysis dashboard, which would collect all of the relevant data and display it in a series of tables and graphs for ease of understanding. These graphs could display things like level start vs completion rates, or a table could show all of the events in the order that they arrive from the game, as well as all of their payloads and relevant data. All of this information would make it much easier for the developers to understand when, why and how things occur within the game, and use this information to balance different parameters to create a fairer gameplay experience.

1.2.2. User Stories

Following our initial research and requirements gathering meetings, and end user consultations, we decided to focus our time and resources on the following streamlined user stories:

- Building a game framework to demonstrate the functionality, and give the developers a foundation on which they can build new levels, test new features, and learn and understand the effects of their changes in a safe but realistic environment.
- Recording at least 12 unique categories of events within the game, and all of their associated data, so that the developers can see key trends within the game, where and/or when certain events tend to occur, and how they influence the frequencies of other events to analyse how fair and balanced the game is.
- Creating a dashboard for developers to view all of the in-game telemetry and analyse it in a much more focused and user-friendly environment. This should give a clear overview of all of the recorded game data so that the developers can quickly spot trends and see where players tend to come stuck, allowing them to quickly understand where and how to make the game more balanced.
- Developing a central database to store all of the events and their respective data payloads, allowing it to be more easily accessed and shared between the other parts of our solution, namely allowing direct communication between the game and the dashboard for real-time data analysis and game parameter updates and balancing.
- Providing 8 unique parameters within the game which can be adjusted manually by the developers, to enable quick adjustments to enemy and tower statistic multipliers to help to dynamically adjust the game's difficulty remotely without manually altering hardcoded values within the game's code.
- Providing full user authentication to secure accounts, with encrypted passwords to keep credentials safe, in order to prevent unauthorised access to he dashboard, minimising the risk of parameters being changed to damage the gameplay experience for players.

- Creating a lightweight simulation to allow the effects of any parameter changes to be measured and predicted natively in the dashboard, and speed up the process of measuring these effects by simulating them instead of playing through the whole game manually.

1.2.3. Sprint 1 Stories and Goals

For the first sprint, we decided to focus on the first four of these user stories, as these would be the minimum to have a working prototype complete by the end of the sprint.

These stories were then formalised into a series of goals for the first sprint, where completion of all of these tasks would signify success in this sprint. In essence, these were our success criteria. These were:

- Two game stages built and playable
- Be able to log in-game events for at least six unique telemetry types
- Have a dashboard with a funnel view showing all telemetry data for game sessions, and three different views in total
- Allow the developer to adjust one parameter in the game
- Have a simulation to show the approximate effect of any parameter changes
- Be able to run five different automated tests to prove functionality and robustness of the system, with documentation on how to run them
- Execute a manual end-to-end test of the entire system, with full evidence of its successful completion
- Produce a full project report to demonstrate and document everything we have produced, and all decisions we made
- Comprehensively analyse all ethical and legal concerns related to the project, notably privacy concerns with data handling and recording, accessibility for all users, and any licensing issues with both our own intellectual property and creations, and any tools, environments or languages we utilised in production

1.2.3.1. Tasks

Having such large user stories meant a lot of ambiguity in how they should be tackled most efficiently, so we decomposed each of them into a series of more manageable steps and tasks, allowing them to be approached more flexibly and meaning nobody had to take sole responsibility of any huge section, encouraging teamwork and collaboration to reach the end goals. These tasks were:

- Game framework
    - Define enemy class
    - Define tower class
    - Define a level and the overall gameplay loop
    - Implement a simple in-game economy
    - Create a simple user interface
- Telemetry events
    - Define events
    - Define how events should be communicated from game through the database to the dashboard
- Telemetry dashboard
    - Create a basic dashboard architecture
    - Implement a system/toolkit to allow developers to adjust in-game parameters
- Central database
    - Have a table to manage all telemetry events
    - Have a table to manage all gameplay parameters

1.2.4. Sprint 2 Stories and Goals

Within the second sprint, the original user stories remained constant. We already had a fully operational game framework, allowing developers to modify game parameters and test their effects, ensuring the game remained fair and fun to play. We had decided early on that giving developers a template to create new levels

was beyond the scope of the project, and would not have been worth the additional complexity, and could instead prove to be a security risk: allowing many developers access to this training framework would cause memory overwrites, conflicts, and other issues that would be safer to avoid. We also could not allow the testing of new features provided by the developer, as this would have the same consequences and complexities. The game is intended to be an example, and not one that the developers should customise themselves and continue to develop. Giving a full integrated development environment was far beyond the scope, and naturally left behind.

As part of this second sprint, we could better approach the second user story: having twelve telemetry types and using all of their corresponding data to show game trends. This was already partially complete from Sprint 1 – there were six defined telemetry types, and through their storage in a database, alongside all of their related data, we were able to produce a series of graphs and tables to show how their frequencies correlated with user experience – if certain telemetry event types correlated with more user level fails, or more players quitting the game, then surely something should be adjusted to make the game more fair and fun to play. This would be easy to extend to the required twelve telemetry types, once we had them all defined, and we could create new graphs to see the effects these types have on the full game.

The third user story was fully covered within the first sprint, but could be significantly improved for the second sprint. We had a fully established dashboard and clear graphs to show key trends, allowing developers to see exactly where key parts of gameplay, and the biggest challenges, were occurring. With more time in the second sprint, we could add more graphs to visualise the data and make it more readable and accessible for developers, though technically this user story was complete.

The fourth user story for Sprint 1 was also achieved in full, with a functional database with a full API to store all of the game's data, and make it accessible from the dashboard to display it all. This could be advanced in the second sprint by adding data validation and anomaly detection, to filter out erroneous data and make the graphs and interpretations more accurate, as well as providing further new graphs to visualise the data and see more clearly where key trends are occurring.

The fifth user story was far from complete at the conclusion of the first sprint, with only a single adjustable parameter provided in the architecture as a proof of concept, but this meant we had the initial framework and database tables to implement everything to complete this user story for Sprint 2. We would only need to define the additional 7 parameters, and then link the game and dashboard via the database again so they could be synchronised, and allow all of these parameters to be remotely adjusted as before.

While we hadn't initially allocated it to the first sprint, we ended up completing the seventh user story. We developed a simulation to show the effects of parameter adjustments quickly and accurately, without playing through the entire game. This made the full balancing suite much more accessible and easier to use, as it provided a swift response to indicate what effect any adjustments would have on the game's difficulty, and what percentage of players would be able to complete any given level. However, this could still be further refined for the second sprint, ensuring it would be compatible with new boss enemies, new levels, new tower upgrades, as well as implementing threading to improving its performance.

For the second sprint therefore, we would focus on the following user stories:

- Offering user authentication
- Providing all 12 telemetry types
- Providing 8 adjustable parameters

These were the user stories which had objectively not been completed by the end of the first sprint, although there were still refinements to some of the other stories which we could yet make, as outlined above.

As a means of having a success criteria to define an ‘end’ to the project, we again streamlined a set of goals which would signify that we had completed our finished product once they were all ticked off. These were:

- Create at least 10 playable levels in total
- Post 12 unique telemetry events to the database after they occur in game

- Provide a full difficulty adjustment suite as the dashboard, with graphs drawing from the telemetry data, and eight adjustable parameters
- Have an updated simulation which could more accurately predict player success, and halve the execution time
- Add balancing rules which suggest parameter adjustments according to estimated player success rates from the simulation
- Make users create their own accounts, with user authentication secured via a password hash
- Test all new features, with documented weekly unit and automated tests and an updated manual end-to-end test
- Update the project report and all other related documentation from Sprint 1
- Create a handover pack to enable the end users to run the project safely and easily

1.2.4.1. Tasks

As before, it was necessary to break down the more complex user stories into easy-to-understand tasks to supplement the success criteria outlined above. Following the conclusion of the first sprint, these were the remaining tasks we had to tackle before the final deadline:

- Telemetry events
    - Define all 12 event types
- Parameters
    - Implement all 8 parameters
- Simulation
    - Provide simulated game agents
    - Optimise the simulation
- Authentication
    - Provide the backend user authentication system
    - Implement authentication in the dashboard
    - Implement authentication in the game

1.3. What We Built

1.3.0. Success

Below, we have outlined everything we built in each of the sprints. To evaluate how close we were to achieving the goals we set out as success criteria, and to what extent we covered all of our user stories, see section 4.3. Evaluation Results. To see exactly what was implemented against each of the goals above, see section 3.2. Requirements and Implemented Features.

1.3.1. Sprint 1 Product

Ultimately, our finished prototype for sprint one comprises three core parts, as was outlined in our original aims and goals. These are the game, database and dashboard.

The game, built in Godot, currently has a main menu, with interactive ‘Play' and ‘Levels' buttons, with ‘Levels' allowing the user to choose from the two available for this Sprint 1 prototype build instead of playing through from the beginning. Within each level, there is a simple user interface, showing the player's current health, their money, all of the available towers and their associated costs, and a play/pause button for the player to commence the level. The towers can be clicked to place them anywhere on the stage, and there are three to choose from, each with different attack patterns. Once the player starts the level, enemies will begin to spawn and move through the map along their route, and the towers will attack any enemies in their range. If enemies make it past all the towers without being killed, the player will take damage. If the player's health depletes to zero, they lose, but if they make it to the end of the stage with health remaining, they win, with relevant ending screens being displayed to them. Throughout gameplay, events are logged, with their relevant data being sent to the database. These events include tower_spawned, money_spent, stage_start, stage_end, stage_fail and enemy_defeated.

The SQLite database, built in Python, receives all of the event data recorded and posted by the game, and stores it in a table. This allows it to be easily sorted and analysed by the timestamps of each event, and the session ID and user ID are logged anonymously to group data to a specific playthrough attempt. For event types like tower_spawned and enemy_defeated, the relevant tower or enemy ID is also returned to the database in the payload, so it can be verified. Then, any game parameters and their values are stored in a separate table, so they can be safely held and updated as needed through the dashboard, with their effects directly translated to the game. Any changes are also justified through a decision log, and these decisions and rationale are held in another table so they can be displayed for accountability and understanding between developers. The database also utilises FastAPI endpoints to connect seamlessly with the dashboard

The Python-based dashboard consists of a number of different functionalities. The home page displays a series of graphs, graphically portraying information like level completion rates and event counts for each event. Then there is a table holding all of the events captured by the game, so they can be clearly and easily viewed by the developer. The decision log holds a record of any changes to parameters within the game made by developers, so any recent adjustments can be viewed and easily understood. The parameters page feeds directly into this, with a list of all parameters that can be adjusted within the game. Developers can alter the multipliers and values, then submit their changes with an explanation and reasoning, indicating the rationale for any changes, so other developers might understand their thought processes and see what has been adjusted. At the moment, we are able to adjust the enemy_damage_multiplier, and any changes are communicated directly to the game in real time, so their effects can be monitored and tested immediately. Finally, the simulation mode uses a random predictive Python script to estimate approximately how many players will be able to win a level with the current set of parameters. This prediction, with a percentage success rate, gives developers a suggestion of any changes they should do in order to more fairly balance the game for an appropriate success rate.

Alongside this core suite of connected parts, we built a full testing system, with unit tests to check the functionality of individual components, and integration tests to validate that every part of the system works together as expected. We have the required five automatic tests, and a guide on how to execute and test these to demonstrate robustness of the system, as well as conducting a manual end-to-end system to prove that everything worked as expected in conjunction, even with user interaction as opposed to seeded test data.

1.3.2. Sprint 2 Product

Expanding on the finished product we had created in the first sprint, we finished the project with a much more comprehensive system. The game, while very primitive but functional in its first iteration, was now much more polished and professional, with improved graphics and assets to make it more visually appealing, but the key change was a whole suite of new features.

Initially, the player encounters a login screen, where they are prompted to input a username and password, and either login to their existing account, or register their details for a new account. We required that all passwords require a mix of uppercase and lowercase, and included at least one number and one special character to make sure user accounts were harder to brute-force. These passwords are all stored securely in the database, with a strong hashing function applied to ensure it s fully encrypted and safe from malicious actors. This login screen also features a disclaimer about our data harvesting, and instructs the user on how they might choose to opt out of data collection and delete all of the data associated with their account. Of course, all of this data collection is necessary to enable all of the dashboard features for improved game difficulty balancing, so it is enabled by default.

Once past the login screen, the player is granted access to the game's main menu, with Play and Levels putting them into the game, and Difficulty allowing them to customise their gameplay experience by choosing from Easy, Medium or Hard. In the bottom right of the menu is a button to log out of the player's account. There is also a settings page which allows them to adjust the game's volume, and there is a button here to delete all of the player's data from the database should the player wish. The Levels menu gives a preview of all of the ten maps available to play, and the player can select any of these to jump straight to that point of the game for an added challenge.

When in the game, the menu and HUD is familiar from the first sprint, but with added features. Towers can now be upgraded: by clicking on them, a new menu appears, with details of the tower's statistics and possible upgrades. If the player has enough money, they can choose to upgrade their towers, giving them a new visual appearance to distinguish them, and enhancing their capabilities: for example, the first upgrade for the Triangle allows it to shoot two projectiles with every attack, while the Square's first upgrade allows it to shoot in more directions. Then, once the player starts the level, the first wave of enemies begins travelling along the path, and the towers begin to shoot to defend the player as before. New for Sprint 2, the enemies have new visual changes, showing ‘cracks' after they have taken damage to symbolise their health declining. There are a number of other new animations, like the enemies breaking once killed, towers recoiling with each attack, et cetera. Additionally, there are lots of extra waves within each level, making each one more involving and more challenging, with a countdown until the next wave arrives, and an option to skip to the start of the next wave. Then the game progresses as before: either the player wins by defeating all of the enemies or surviving with health remaining, progressing to the next level, or they lose by taking too much damage within the round. In later waves and even more so in later levels, boss enemies will appear, with significantly more health and dealing much more damage than regular enemies. They take a concerted effort to combat, making the game challenging and involving to complete successfully. They also provided three new telemetry types: boss_start, boss_fail, and boss_defeated, to indicate how much trouble players were having when it came to defeating these new, difficult enemies. We also added a stage_quit telemetry event to indicate where enemies had rage-quit the game, as this would pick up any trends of prohibitively difficult levels that had to be immediately targeted for balancing adjustments.

The database remains much the same since the first sprint, except the additions of a balancing rules table, which calculates whether or not an adjustment to the parameters should be suggested based on a series of minimum and maximum values for things like win rates, et cetera., and the encrypted password tables and usernames, now that we have successfully implemented user accounts and authentication. The API has also been expanded with the additional seven parameters and six new telemetry types, so these tables have grown to accommodate the new changes. The dashboard could now submit adjustments for all eight parameters, and the game would pull these changes to vary the parameters and alter the difficulty dynamically. We also provided a seeding script to populate the database with realistic game data, enabling the analysis of the graphs and telemetry, and to look out for any data anomalies which would provide incorrect or impossible events. We aimed to filter these out wherever possible, to ensure the data was as accurate and real as possible.

Within the dashboard, there are also a number of new changes and features. Like the game, developers now encounter a login screen, where they can either sign up with a new account, or log in with existing credentials. The same credentials for a game account can be used, as the accounts are shared throughout the entire system, but only approved developer or admin accounts can access the dashboard, to prevent malicious changes to sensitive game parameters. When logging in with an unverified account, users can press a button to request admin access to the dashboard, sending a request into the dashboard. Existing admin accounts can view pending requests in a new page of the dashboard, and choose to approve or decline them. Developer accounts can access the game and the dashboard, but unapproved accounts only have access to the game.

Once successfully verified and logged in, the dashboard now features lots of new elements. As part of the user interface across every tab, there is an Accessibility option, where developers can alter settings to make the interface easier to use and view for those with visual or audible impairments. There is also an option to export all of the information available on the dashboard should the developers wish to do so.

On the home screen, there are two new graphs: an enemy difficulty heatmap, showing where enemies are causing the player the most trouble, and an economy balance chart, indicating where most of the player's money is being spent, and how effective it is. The dashboard and decision log tabs are the same as before, just with new telemetry types appearing in the game data supplied on the dashboard page. The parameters tab has been updated to show all eight of the adjustable values, with default multipliers set to 1.0, but all are able to be altered at the developer's discretion.

New is the Balancing tab. This takes key values like player win rates, as calculated from the most recent run of the simulation, and according to the balancing rules, suggests action to alter the parameters and make the game more fair. For example, if the player win rate is too low, the dashboard will suggest decreasing the enemy damage or health multipliers, to make them easier to defeat and raise the overall player success rate. All of the balancing rules are purely based on the hard data from the simulation, and do not implement any prediction or artificial intelligence in their suggestions.

The simulation tab is unchanged, aside from a new difficulty selection drop-down, so the developer can run the simulation to see the approximate effects on each of the three difficulty levels. However, in the backend of the dashboard, the simulation scripts have been substantially improved, to include new enemy hit-masks, improved precision, and significant enhancements to the time taken for these scripts to execute. Previously, they could take as much as three seconds for every test, which meant it was prohibitively slow for the large number of tests required to get a truly accurate average value, but now, ten test runs will complete in as little as a second, marking a huge improvement, allowing much higher numbers of tests to be completed in a fraction of the time to get quick and accurate results to enhance the developer's ability to balance the game.

The next new feature is the Anomalies tab. This searches through all of the game's telemetry data stored in the database, and looks for any impossible sequences of events, or other errors in the system. These include a level completing before it has started, enemies killed before they have spawned, or towers being placed outside the map's bounds. These kind of events are logged in the anomalies tab for the developers to review, with suggestions of what to check or modify in the game or data validation scripts to prevent further glitches of this type. It also has a button to manually run an anomaly detection script, and one to clear all anomalies found, and wipe this erroneous data.

The final tab of the database is Admin Requests. As alluded to before, developers who wish to access the balancing dashboard must first have their account verified as such, and any admin permission requests will appear here, in the new dashboard tab. That way, any existing developers or admins can assess these requests and appropriately approve or decline them, keeping the dashboard secure with only authorised users having access to it.

1.4. Outside the Scope

Due to time constraints as the project neared its deadline at the end of Sprint 2, we had to limit certain elements of the development process in order to be able to produce a fully finished, polished, functional application. While we made every effort to stick to the original scope we had set out at the beginning of the project, we understood that it would be significantly more important to have everything in working order, than risk volatility as a trade-off for greater functionality. As a result, there were parts of the game which we were keep to work on, but had to leave outside the scope for this project in the allocated timeframe.

The first of these was a potential ‘endless mode’ for the game. As it was developed as a wave-based tower-defence style game, it would have been theoretically possible to produce an alternative game mode, differing from the usual level-based progression system, in which the game infinitely and pseudo-randomly generated new levels and waves based on a seeding system that we would provide, and an algorithm for generating new levels. This would give a lot more functionality to the game, with more ways to play, and a chance to retain player interest long after they had completed the limited number of levels produced in the normal game mode. Ultimately, this pseudo-random generation algorithm would have been too complicated to develop in the limited time we had available. Additionally, this would have created significantly more work on the database and dashboard, with new telemetry types and data recording structures needed, since the levels and waves would be infinitely generating, and this would also require much more rigorous data validation, with the range of permitted values now ranging from known constants to infinity, and despite pledging to provide this endless mode at the Sprint 1 review, we deemed that we should limit additional development on the game, and focus on the core part of the system as far as the requirements were concerned: the database and dashboard.

This focus on the dashboard side of the project again caused us to adjust our targets and development decisions within the game. Another feature we discussed at the start of the second sprint in our initial planning

meeting, but was ultimately left outside the scope for the sprint, was some type of reinforced enemies. Our idea was to have some proportion of the enemies in certain levels as reinforced enemies, perhaps with ‘armour' which would give them a chance to deflect all projectiles aimed at them, making them significantly more challenging to defeat, for a more engaging and challenging gameplay loop in the levels where they might appear. While we had developed the assets for these new enemy types, we deemed it too time-consuming to make it worth developing in this sprint, and it would have had little effect on the finished product, especially when the core part of the project's success centred on the dashboard. However, by making the decision to move this aspect out of scope early in development within the sprint, we were able to reuse these new assets, instead adjusting their size and using them as bosses within the game, giving them a unique appearance within the game to signify their significance.

One key element which concerned both the game and the dashboard was the ability to customise the waves of enemies throughout the game. Since the tower-defence game design allowed multiple waves of enemies in each level, this offered up lots of opportunities to customise the difficulty levels in the game. As part of the balancing suite that we provided in the dashboard, we considered allowing the developers to manipulate each of the individual waves in the game, or at least the number of enemies that would appear in each wave. Theoretically, this would be a very dynamic way of ensuring the game was fair and balanced, as it would allow the developer to reduce the number of enemies faced by the player if the level were too difficult and vice versa, and this factor could be customised uniquely for each of the three difficulty levels. Taking it even further, the developer might have been able to specify exactly how many of each enemy type should spawn, how many bosses or reinforced enemies (out of scope, see above), and we would provide new balancing rules to add to these possibilities. Much more data analysis would have been required, as the balancing rule suggestions would need additional telemetry to see which enemy types were causing the most player fails, and thus how to fairly adjust the number of each type without making the game prohibitively difficult, et cetera. The possibilities for customisation within the game, and being able to give developers even more power and independence over the game, were very exciting, but ultimately the extra development required to remotely alter the hard-coded wave details through the dashboard, and to aid the developers with balancing suggestions, were far too challenging to attempt within this sprint, but would be a very enticing prospect should we be able to develop the system further in future.

2. Design and Related Work

2.1. Early Prototyping

Prior to beginning development work on the system, we considered all of the tasks we had isolated, and all of the requirements we would have to implement into the finished product. Based on this task analysis, we created low-fidelity paper-based prototypes of what the game and dashboard interfaces might look like. For simplicity and abstraction, these were devoid of any depictions of gameplay in the case of the game prototypes, and no simulated data was provided as an example in the dashboard prototypes. These paper demonstrations gave an overview of what every menu or interface might eventually look like, with all of the necessary graphical elements outlined, alongside a series of detailed notes explaining the rationale for every decision, such as colour, shape, font et cetera., with a focus on accessibility to ensure the system would be compatible with users with visual impairments or reading difficulties. They also had an explanation of every element included on each screen and how they might be interacted with, to show what they are used for and how they could be integrated. Finally, we made clear outlines of the natural progression through the menus and interfaces, to show the order in which the different views and screens might be displayed, and how they would be accessed.

2.2. Architecture and Dataflow

Our overarching architecture consists of a tower defence style game, which posts event data to a database, which in return is accessed by a telemetry analysis dashboard which displays all of this information in graphs

and tables. In the opposite direction, the dashboard sends parameter information to the database, which is then read by the game, and changes to parameters are implemented in the backend, affecting gameplay aspects.

- As such, during gameplay, data is constantly flowing between all three core elements of our system, with the game posting telemetry event details to the database every time any key detail occurs, such as damage being dealt, stages starting or ending et cetera.
- Alongside a notification of the event occurring, with a timestamp, user ID and session ID, the post will have a payload containing other associated data, such as enemy ID for enemy_defeated, and other similar bits of information which the developer can use to analyse the gameplay.
- This data is permanently stored in the telemetry section of the database API, and this is retrieved by the dashboard and displayed in a more readable format in the telemetry page of the dashboard.
- The dashboard also manipulates all of this information, extracting stage_start and stage_end/stage_fail events to create a graph demonstrating player level completion rates. Additionally, every event type is counted and their total individual frequencies are displayed in another graph.
- The game's adjustable parameters are also stored as part of the database API, with the game fetching any constants like multipliers before each gameplay session to ensure the most up to date and fully balanced version is being played. The database receives these changes from the dashboard, where they are altered via the parameters page.
- Submitted alongside any parameter changes are rationale and evidence for each adjustment, both of which are sent to be stored in a third table within the database API.
- The database also holds a table of all user details, which are first input from either the game or the dashboard, wherever the user first creates their account; then are referenced each time a user attempts to log in, meaning lots of dataflow between all parts of the system.
- The database also holds further details of balancing rules and data anomalies, all of which are in constant use throughout the dashboard.

As part of our architecture, we also decided to deploy each of the three core components as an independent Docker unit, so that they are much more portable, are easy to run on any system, and they can run independently even if an error occurs in another component, keeping more of the system functional at all times.

2.2.1. Rationale

The compromise of added complexity designing the Docker containers was able to be mitigated by the associated benefits, so as such, our architecture, while in an online deployment like our demo is very much tightly integrated with heavy dataflow between components, it is also possible to isolate each of them. This decision helped us to all access and understand the system, rather than relying on more powerful machines or the server, giving us more flexibility in design and development.

Our rationale for the overall architecture centred around the central database. It was clear from the project specification that we would need to independently develop both a game and a telemetry dashboard, but the means of communication between them was flexible. By choosing a database, we are able to store all of our data permanently, rather than having to reset the data every time the game or the dashboard is launched on a new device or session. Alternatively, we could have stored the data in either the game or the dashboard, but with thousands of data entries, this would have consequences on file size and execution speeds, so this was not an option. Since we had access to a remote server, we decided that storing all of the data to be fetched later on would be by far the most efficient option, and this would reduce the overall dataflow as the data would only need to be retrieved once, with new changes being the only communications between the components while in operation. Writing our own database API also gave us full flexibility to store our data in easily manipulated and understood tables, making data transfer considerably easier.

2.3. Similar Approaches

As outlined above, there were a couple of alternative approaches to the architecture of our system, though all were similar in their inclusion of an independent game and dashboard. In contrast to our database solution, we could have stored the session data in the game or the dashboard, though for Sprint 2 this would be unacceptable, as we would need a way to log thousands of data entries for analysis, and this solution would not facilitate this. Another option would be to write all of the data entries into a .txt or .json file. Though this would theoretically work, the problem of portability would get in the way, as these files would have to be remotely accessible to any developers who wanted to see the telemetry events and adjust parameters – sharing would be a big issue, and ultimately, circumventing all of these issues with a remote server made the process considerably easier.

We could have opted for a platformer style of game, but due to the gaming experience of team members with similar styles of game, a tower defence genre was settled on, as familiarity would make it easier and more fun to program, making the whole program smoother. In addition to this fact, more objectively, tower defence games are completed strictly linearly, with a fixed time duration of each level, as all of the enemy spawns are predetermined in each stage. This makes difficulty analysis easier, as trying to interpret the time duration on a platformer level could throw anomalies into the data; though intuitively a longer play time could show a more cautious and inexperienced player, it could also be interpreted from a player attempting some form of challenge, or exploring for secrets, or indeed not playing at all; these misinterpretations could affect genuine attempts to balance the game, by skewing certain telemetry events. Thus, our choice of tower defence game would give developers a better chance to accurately predict player behaviour and balance the game in a true fair fashion.

Indeed, we feel that we chose the most ideal and applicable approaches to the given scenario; though evidently we could have made different decisions with the architecture, or even down to our whole game concept, we firmly believe that our approach was the best option for our team and the resources we had.

2.4. Scientific Advances

Our project centres on the idea of ensuring games are fair and balanced from all players' perspectives. If we can achieve this with our use of data analysis, this may indeed have useful scientific implications, as our simulations and tests could indicate the effects of changes on a system. This can be expanded far beyond the scope of the video gaming industry, with potential applications across science, medicine and more: if a simulation of an independent system can be created, with a series of adjustable parameters within it, then certainly the effect of any changes might be suggested and predicted by the simulations before deploying the changes directly into the system. This can substantially reduce risk, as having a clear idea of what is expected to happen would reduce the possibility of risky or reckless changes being deployed, minimising damage and cost, helping to arrive at an accurate solution quicker and safer.

3. Implementation, Methods and Materials

3.1. Key Components

3.1.1. The Game

The game as a key component comprises a full graphical suite with interactive features, providing an engaging and interactive gameplay experience.

3.1.1.1. Menus and User Interfaces

Upon loading the game, the player is met with a login menu, with an opportunity to create a new account or sign in with an existing one. There is also a disclaimer about data collection to ensure GDPR compliance. Within the game is a central main menu, from which the user can choose to begin from the first stage, by pressing ‘Play', or to customise their start point, choosing the starting level from ‘Levels'. Within ‘Levels', a

depiction of each map is portrayed, so the player might be reminded which is which, or choose based on personal preference. The main menu also allows players to choose their preferred difficulty level and check the how-to-play guidance, while the settings menu allows players to change the volume, as well as opt out of data collection and delete any existing data collected.

In the gameplay loop, once the user has chosen a level as a start point, the user interface (UI) and heads-up display (HUD) offer information and points of interaction. In the UI is a play arrow in the top left, allowing the user to manually start each stage after they have placed their desired towers. There is also an option to skip to the next wave while the level is in action. On the right of the screen is a depiction of all of the available towers, and their costs. The player can click these and place them on the game map. The HUD gives an indication of the player's current money, so they can calculate how many towers they might afford, as well as a health bar to indicate when they have taken damage, and how close they are to stage failure. Each tower can be clicked to bring up its upgrade menu.

At the end of each stage, one of two outcomes is reached. If the player was able to defeat all enemies, they are shown a win screen, and from here they can click 'Main Menu' to return to the initial screen, and decide where to progress. However, if the player took too much damage and lost all of their health, they are shown a loss screen, which provides the option to return to the main menu, or 'Replay Level' if they wish to attempt it again.

3.1.1.2. Towers

The core gameplay concept is to place defensive towers around the enemies' path in order to kill them all and protect the player's health. Currently we have three different towers, each with unique implementations. The triangle shoots a single projectile at a time, targeting the enemy within its range which has progressed furthest along the path. The square fires projectiles from each side, not necessarily targeting any particular enemy. The star deals damage over time to every enemy within its range at once. These each have different costs, so the player must be strategic in their choices, as well as their placements, finding the optimal location for each tower to be most potent. The towers each have an upgrade path, with the player being able to spend money to upgrade each placed tower, giving extra perks to make them more powerful and help the player in defeating all enemies.

3.1.1.3. Enemies

We have five different enemies which may spawn within the game's stages. The number and type of enemies is predetermined for every level, with the intention of providing unique challenges and gameplay scenarios each time, forcing new strategies from the player. Each level has progressively more waves to make late-game more challenging. The enemies are all circles, but in small, medium and large sizes, with differing attributes to go with this. There are also two unique boss enemies, with armour-plating, much more health, and a substantially larger size. Every fifth wave is a boss wave, giving a new challenge for the player to take on. It also has enough damage to defeat the player instantly. The larger the size, the slower it moves, but the more health it has, and the more damage it deals.

3.1.2. The Database

The database implements its own API to dictate exactly how all of the system's data should be stored, making retrieval by either the game or the dashboard much simpler. Below will outline the full structure of each data entry and table.

3.1.2.1. Telemetry

```
Telemetry (
        id: Integer {PK}, user_id: Integer {FK ref Users(id)}, stage_id: Integer, telemetry_type: String,
dateTime: DateTime, data: JSON (Optional)
)
```

3.1.2.2. Parameter

```
Parameter (
        name: String {PK}, value: float
)
```

3.1.2.3. BalancingRule

BalancingRule (
id: Integer {PK}, trigger_condition: String, suggested_change: String, explanation: String, (Optional)
)

3.1.2.4. DecisionLog

DecisionLog (
decision_id: Integer {PK}, parameter_name: String {FK ref Parameter(name)}, stage_id: Integer (Optional), change: String, rationale: String, evidence: String, (Optional), dateTime: datetime
)

3.1.2.5. Users

Users (
id: Integer {PK}, username: String {unique}, password_hash: String, is_admin: Integer {default=0}, created_at: DateTime, is_requesting_admin: Integer {default=0}
)

3.1.2.6. Anomalies

Anomalies (
id: Integer {PK}, anomaly_type: String, resolution: String {optional}, created_at: DateTime
)

3.1.2.7. Telemetry Anomalies

TelemetryAnomalies (
telemetry_id: Integer {PK, FK ref Telemetry(id)}, anomaly_id: Integer {PK, FK ref Anomalies(id)}
)

3.1.3. The Dashboard

The dashboard is the single most important aspect of our project, as this is what allows us to provide the full balancing toolkit to developers, so that they are able to analyse the data and adjust parameters to change the gameplay experience. To aid the developers, we have produced a series of different tabs and views to offer different features and a more comprehensive and intuitive suite. On all pages are a logout button, and an option to export the data on the dashboard. Before the developers can access the dashboard, they are prompted to sign up with a new account, or log in with an existing one. This will have to be with an account with admin privileges – see 3.1.3.8. Admin Requests.

3.1.3.1. Home

The initial homepage of the dashboard features the key graphs and funnel views for the data, giving a visual representation of some of the core statistics elicited from the data sent by the game. These include telemetry type counts, offering a summary total for each of the telemetry types recorded; drop-off points, indicating the number of stage starts for each of the levels, potentially revealing at which point most players give up the game; another graph showing the proportion of stage starts against stage finishes, indicating any players who quit mid-level, and thus where the levels may have been too difficult; an enemy difficulty heatmap, showing what affects the rate of enemies defeated; and an economy balance chart demonstrating how players are using their money.

3.1.3.2. Dashboard

The dashboard tab is effectively a more organised and readable view of the telemetry table from the database. It shows every single recorded event, sorted by the time they were recorded, and indicates the anonymised user ID, which stage it occurred in, the actual telemetry event recorded, and the associated data payload. The developer can then manually read through these event logs to see a timeline of where things happened during a particular run of the game.

3.1.3.3. Decision Log

The decision log is a history of all changes made to different parameters within the game. It has a record of the evidence and rationale submitted as part of the alteration, so this log can be used for accountability, verification and learning. It can help to see previous values and their perceived effect, so future adjustments can be fine-tuned between similar values which didn't quite reach the desired balance of difficulty.

3.1.3.4. Parameters

The parameters page shows all of the parameters within the game which can be adjusted in order to affect the difficulty and balance the game more effectively. There are eight adjustable parameters, so the game's difficulty can be dynamically altered. Once the constants have been changed, the developer can save their changes, and will be prompted to provide relevant rationale and evidence for the changes, for accountability later on. All of this will then be logged in the decision log.

3.1.3.5.Balancing

The balancing tab calculates a range of values based on the data provided from the most recent runs of the game, and uses these to assess whether or not changes are needed in terms of the game's difficulty. There are six implemented balancing rules, whereby if a certain value is too high or too low, action is suggested to the developer, indicating where balancing is required. Alongside this, a specific parameter is proffered to the developer, which might provide the desired balancing change.

3.1.3.6. Simulation Mode

The simulation mode allows the developer to run a simple simulation of the game to predict the effect of any changes to parameters, and see approximately what proportion of players are expected to complete a given level on a chosen difficulty. The simulation works by randomly generating tower placements near the enemy path, then seeing if that tower configuration is capable of winning the level or not. This random generation is intended to simulate the naturally unpredictable nature of human players, though might be refined to favour certain areas once we have more player data. This is intended to significantly reduce the time it takes to test the effects of parameter changes, as opposed to manually running the full game, and it reduces developer bias by being pseudorandom. The more simulations that are conducted, the more accurate the estimate of the change's effects. Alongside a predicted percentage is a suggestion on whether or not any other adjustments are needed based on this.

3.1.3.7. Anomalies

The anomalies tab searches through the data collected from the most recent gameplay, and checks for any impossible events or sequences occurring in the telemetry. For example, if an enemy is killed before it spawns in, or if a level is completed before it has started, et cetera. These glitches within the data will be extracted and displayed on the anomalies page, all collected under anomaly categories to be easier to understand. There is also a button on this page to purge all anomalous data, and wipe it from the database to make the graphs and tables more accurate.

3.1.3.8. Admin Requests

When a user attempts to log into the dashboard with a non-admin account, they are prompted to request admin access. When they do this, a request appear in the admin requests page of the dashboard, and can be seen by any verified developers with existing admin privileges. They can then choose to approve or deny the request at their discretion.

3.2. Requirements and Implemented Features

3.2.0. Requirements

For a full outline of the requirements and goals defined for each sprint, see sections 1.2.3. Sprint 1 Stories and Goals, and 1.2.4. Sprint 2 Stories and Goals. To see an evaluation of how well the features implemented below hit these success criteria, see section 4.3. Evaluation Results.

3.2.1. Sprint 1 Features

From the goals consolidated from the user stories and requirements gathered at the beginning of the project, these were the features we implemented over the course of Sprint 1 to cover all of these success criteria.

- Two game stages – we developed a basic framework for the game levels, which could be quickly and easily designed and implemented as a path for enemies to follow. The first two of these were fully designed and pushed to the prototype build for this Sprint, with enemies moving as intended, able to be killed by towers placed by the player, and the player could both win and lose, as demonstrated in

the presentation in Sprint 1. They are also playable at www.game.mm25564.uk (defunct as of Sprint 2 conclusion).

- Logging in-game telemetry events – the game would send a data payload to the database every time certain events occurred. We implemented eight of them initially: stage_start, stage_end, stage_fail, enemy_defeated, damage_taken, money_spent, tower_spawned and tower_upgraded, with tower upgrades not yet implemented but a target for early in Sprint 2. These would all send related information to the database, like a payload with details of the user_id and which stage the event occurred on, plus anything relevant to that event, like the amount of money_spent.
- Dashboard with three different views – at www.dashboard.mm25564.uk (also now defunct) is the telemetry analysis dashboard that we built, with three unique graph views: Stage Start vs Finish, showing the proportion of players that completed each level successfully for every time that the stage was started; Drop-Off Points, indicating how far along the players got before quitting the game (not so useful with only two levels implemented, but with the full suite at the end of the project, this will give a clear indication of where players are losing interest and motivation); and Telemetry Type Counts, showing exactly how many of each event type have occurred.
- Parameter adjustment – within the dashboard is a page which will list all of the adjustable parameters in the game. Next to the parameter name, which should be concise enough to explain exactly the purpose, is a value field where the developer can alter the multiplier, and this change, once submitted, is sent to the database parameter table, and all of these values are pulled by the game before each session, ensuring the effects can be seen live and the game is kept fully up-to-date with the most balanced version available. For now we have implemented the first of these, and enemy_damage_multiplier, and it works as intended and outlined above.
- Simulation – another tab in the dashboard offers a lightweight simulation, implemented as a Python script, which attempts to predict what proportion of players will be able to complete any given level. It takes the chosen level, and the number of tests to run as inputs, and returns a percentage of those tests that returned a win. It randomly selects tower placements and estimates whether that placement configuration is capable of killing all of the enemies, or if the player will lose. This is also dynamically affected by parameter changes, so developers can quickly judge whether their changes have the desired balancing effect.
- Five automated tests – we designed a series of automated tests to ensure different components of the system were robust and reliable. The details of all of the results of these tests are detailed in the Testing Evidence file, showing tests across the database, dashboard, and the API, to ensure all was working correctly, and showing what potential outcomes could be expected for every test, with both success and failure eventualities accounted for.
- Manual end-to-end test – also in the Testing Evidence file is the detailed instruction of how to execute a full manual end-to-end test of the system. It was designed to show that all parts of the system are working together in tandem without errors or glitches, and is divided into steps with details of how to ensure everything is working as expected and intended, and what may have gone wrong at any given stage.
- Project report – this document was written to show the full process of our project. It aims to cover every aspect of the project and every step we took along the way, with the design, implementation, risks, mitigation, evaluation and contributions all outlined, with key decisions justified and their impacts made clear.
- Ethics and legal – we had to consider a number of potential ethical and legal risks throughout the project. These included: data protection, which we minimised in this sprint by anonymising all user details by not providing user accounts yet; license decisions and compliance, which we settled on an open source license for, which matched the terms and conditions of all of the software we used throughout; and a number of other risks outlined in the Ethical and Legal Considerations document.

3.2.2. Sprint 2 Features

Following the conclusion of Sprint 1, we built on what we had, to polish off the finished product, as well as implementing a series of new features as outlined below, and according to the goals and success criteria constructed at the start of Sprint 2.

- Ten levels built – building on the framework model we built in Sprint 1, we produced another eight new maps, all unique in their design. They are all playable at www.gamev2.mm25564.com. We also made significant improvements and variations to the number and design of the waves of enemies, to make each level progressively more challenging, and to keep the gameplay interesting and fresh. We added tower upgrades, so the player can make more progression throughout the levels, and introduced boss enemies as a new challenge in later levels. Players can also now choose from three difficulty options to customise their own experience. We also added sound effects and animations throughout the game to make it more fun and immersive.
- Twelve telemetry events – with the first eight defined in Sprint 1, and new features implemented in the game as above, we added four new telemetry types: boss_start, boss_defeated, boss_fail and stage_quit. The boss-related types are useful for seeing if these new powerful enemies are too challenging, or are causing too many players to fail, while stage_quit is a new stage related variant, indicating where a player quit mid-level, as a rage-induced quit as opposed to a win/loss in that level. All types are correctly sent via the database and fetched and displayed in the dashboard.
- Difficulty adjustment suite – the dashboard, now hosted at www.dashboardv2.mm25564.com, has been comprehensively upgraded, with new graphs to show relative difficulty, and most importantly, all eight adjustable parameters. These are enemy_damage_multiplier, enemy_speed_multiplier, money_earned_multiplier, easy_health_multiplier, medium_health_multiplier, hard_health_multiplier, triangle_radius_multiplier, and star_radius_multiplier. The latter two alter the damage radius for the triangle and star towers, to make them more powerful, while the three health_multipliers adjust the relative player health in each of the difficulty levels, so they can each be balanced independently of each other.
- Updated simulation – the simulation scripts have been developed with more accurate and advanced hitmasks, ensuring that it is as accurate as possible in its predictions of how many players can complete each level. It has been scaled with the new levels, adapted for the new adjustable parameters and difficulty levels, and its performance has been improved more than tenfold, with ten tests now taking just a second to execute.
- Balancing rules – a new table was introduced in the database, holding a series of rules whereby if a certain value, like player success rate, was above or below a set threshold, the developer would be prompted to adjust the parameters to better balance that level, with suggestions as to which parameters could be hit to have the desired effect on overall difficulty.
- User accounts and authentication – we have implemented user accounts across all parts of the system, prompting users to create accounts on their first visit to the game or dashboard. All user passwords are hashed and encrypted to ensure they are safe, offering robust user authentication. Also, to protect the dashboard and the game parameters from malicious actors, we separated player and developer accounts, with developers requiring verification by an existing admin account before they are granted access.
- Testing – we continued to produce automated tests to ensure the system worked as expected, and updated the end-to-end test as new features were implemented, to ensure it still gave a sufficient run through all of the important elements. We conducted weekly tests as new features were pushed to the prototype, to establish that nothing had broken with each iteration, and everything still worked together across all parts of the architecture.
- Updated project report – this document was substantially evolved for the second sprint to show all of the differences between the first and second sprints. This mainly focused on new features, and risks, but another integral part was implementing all of the feedback received in the first sprint, to better clarify requirements and success criteria, as well as our evaluation as such.

- Handover pack – as we approached the end of the project, and the handover of our finished prototype to the end users, we had to conduct user surveys to ensure that the product we had created met the expectations and requirements of the users before we handed it over. We also had to make sure these users would be able to maintain the product, so we produced maintenance and troubleshooting instructions alongside a basic user manual for operating the system.

3.3. Tools

The game is built in Godot, as this allowed easier graphical user interface (GUI) implementation, and simplicity with deployment. The database is SQLite based, with FastAPI and Jinja2 endpoints, and is built in Python. It implements its own API, with tables for telemetry events, parameters, parameter changes, and eventually will also feature balancing rules. The dashboard is built with Python scripts, including the simulations, but with HTML and CSS for the online web-based deployment. It also uses JavaScript modules like Chart.js to generate the graphs and display this data graphically. The user account authentication is secured through HTTPS, JavaScript Web Tokens, and an Argon2 hashing algorithm. All online parts of the system are hosted on a remote server, with specific web addresses which are kept online during deployment for users to access the systems. All three core components (game, database, dashboard) are also deployable as individual Docker containers.

3.4. Data Analysis

Our aims for data analysis in this project are limited to supporting developers in improving balancing in their games. The team used visual and mathematical data analysis methods to achieve this goal. Raw data obtained from the game must be processed/analysed in order to be useful to the front-end users, as raw data itself will not give any value to a developer who wants to see potential drop-off points and/or anomalies in gameplay. These are very important points for any balancing endeavour, which can only be achieved through rigorous, yet precise data handling.

3.5. Analysis Methods

The data acquired is analysed through visualising via graphs and numerical representation of relevant details. The team utilised modules such as Chart.js in order to display relevant and useful data that may help developers with improving balancing within the game. Furthermore, the simulation is designed to analyse the game mathematically to produce a final verdict about the playability of the game. The simulation uses statistical methods such as the normal distribution, randomisation and percentage calculations. The simulation is also very scalable, making it ideal to improve upon for the next sprints.

To expand upon the visual analysis of data talked about in 3.3. Tools, drop-off points as a funnel view is an adequate method of data analysis, which we have relied upon in the current sprint of the project. By using a comparison of completion rates of each level, side-by-side, in funnel order, we can clearly see the rate of descent of player successes, which clearly shows if the difficulty of each level and the game as a whole is balanced.

4. Testing and Evaluation

4.1. Tests

We used a Test Pyramid model for our core approach, which is a way of structuring tests to maximise efficiency and reliability. This includes executing lots of unit tests, some integration tests, and minimal end-to-end tests, hence forming a pyramid structure.

Conducting lots of unit tests provided fast feedback, and highlighted any errors quickly. For these unit tests, we included:

- Database model tests

- Individual API endpoint tests
- Error handling

The integration tests involved monitoring interactions between the dashboard and the database API to ensure these worked as intended in operation, as well as accessing data from the database.

The end-to-end test is used to demonstrate that all parts of the system are working together effectively, communicating as intended, and showing what everything does. It begins with familiarisation of both the game and dashboard, before playing the game, monitoring telemetry from the dashboard, and concludes with altering parameters via the dashboard and seeing the real-time updates in the game again. This is useful for seeing how real users might use our system.

Overall, the database and dashboard aspects were very well covered by our tests in Sprint 1, but one key limitation was the lack of automated tests for the game. We conducted lots of playthroughs to make up for this, but there was still potential for hidden bugs that regular gameplay might not highlight. Another weakness in the first sprint was the lack of UI testing in the browser version of the dashboard, but this was again manually tested.

For Sprint 2, we increased the number of tests across all parts of the project. We began conducting weekly tests to cover any new features implemented week-on-week, and fully documented all of these with the outcomes and any glitches and fixes found. The automated tests were also updated to match significant system changes to ensure they would continue to run as expected, and they were executed regularly to ensure normal running of the system and find any bugs introduced with each iteration of the prototype.

4.1.1. Metrics

The majority of the automated tests checked for an output of a status code to show that the desired operations had successfully executed. This would then be totalled for each automated test package, and a percentage of successful results would be returned with a summary of all of the tests and which ones succeeded or failed. These automated tests fully verified the operation of the database and dashboard, including database operations, foreign key relationships, dashboard rendering, error handling and JSON validation:

- Database tests: 2
- API tests: 8
- API authentication tests: 9
- Dashboard tests: 8

Total: 27

We also considered how well the tests covered the code, and tried to evaluate the extent of our code coverage to ensure that we were checking as much of the system for bugs as possible.

4.1.2. Results

The table below outlines our code coverage success:

Name	Statements	Missed	Cover
dashboard/app.py	197	99	50%
database/constants.py	1	0	100%
database/main.py	183	20	89%

In the final version of the prototype, all automated tests passed successfully, however during development there were a few issues. Full evidence of the tests passing is detailed in testing_evidence.pdf, but all automated tests can be executed, and should produce the same results.

Within the dashboard tests, during development, we came across a few errors and glitches. Some of the tests failed as the mock requests.get functions were not replicating the behaviour of the real requests.Response object. This resulted in false-positives and false-negatives, as 404 errors were thrown when the routes were

not correctly mocked, and 200 response codes returned when errors were expected. The solution was to create a custom FakeResponse class that accurately mirrored:

- status_code
- ok
- headers
- json behaviour

This improved the reliability of the tests significantly, offering more accurate results.

4.2. Evaluation Approach

In our evaluation of how successful we were across this project, we focused on the user stories and goals defined in sections 1.2. User Stories, 1.3.1. Sprint 1 Stories and Goals, and 1.3.2. Sprint 2 Stories and Goals. If we had been successful, we would have covered all of the goals set out across these sections, and all of the user stories would have been comprehensively covered.

To better assess this, in addition to a self-evaluation of how many of the goals we had completed, we also surveyed end users to ensure that the product was easy to use and matched their expectations. If we had truly succeeded, the feedback would be overwhelmingly positive, with no gaps found in our finished prototype. However, if we had missed anything that left end users dissatisfied, then we could not have been completely successful. We checked with a focus group of game players, and a handful of developers, to get every opinion on all aspects of our final product. The game survey assessed how difficult the players found the game across all difficulty levels, before we made some parameter changes in the dashboard, refreshed the game, and made the players attempt the game on all difficulty levels again to see if our parameter adjustments had successfully made the game more balanced. It also checked the usability of the game's login pages and menus to ensure players found it easy to navigate and understand. The developer survey investigated whether or not the system was easy to use, and offered all of the features they expected in the package.

4.3. Evaluation Results

We self-evaluated to what extent we had been able to hit all of our goals, with the full details of what had been included for each goal outlined in 3.2. Requirements and Implemented Features. To this end, we felt that we had been largely successful, with everything that we had initially targeted implemented successfully. There were areas that we had been forced to leave behind, as explained in 1.4. Outside the Scope, but none of these had been core to our goals.

In terms of our questionnaires, we found that the majority of players had minimal challenges navigating the menus and user interfaces, with the login page being very easy to navigate, and the rest of the game similarly easy to operate and use. On the first iteration of the gameplay, users generally found ‘easy' mode to be a little challenging, ‘medium' on average quite difficult, and ‘hard' was nearly impossible. We returned to the dashboard and fine-tuned the parameters, and on returning to the game, ‘easy' mode was much easier for players, perhaps too easy, ‘medium' had a clear Bell-curve with a perfectly balanced difficulty rating, and ‘hard' mode being difficult, but now much more approachable; this was clear evidence that our dashboard was successfully able to balance the game as intended, with obvious changes to how hard the game was perceived by users.

Difficulty	Mean - Before Balancing	Mean - After Balancing
Easy	3.37	1.95
Medium	3.79	2.68
Hard	4.16	3.58

The majority of this assessment was shared by the developers we surveyed, with most finding the dashboard easy to use, functional and useful in its features and implementation, and capable of achieving the necessary ability to change the game's difficulty. However, it did highlight one feature that we had missed as a consequence of scope drift. Outlined in our original user stories, gathered in our initial consultation, was:

- Building a game framework to demonstrate the functionality, and give the developers a foundation on which they can build new levels, test new features, and learn and understand the effects of their changes in a safe but realistic environment.

We had successfully produced a game framework, and developers were able to understand the effects of their balancing changes safely, helped by the simulation mode, and user authentication ensuring data integrity. However, we had missed the desire for developers to be able to build new levels and features. This would have been exceptionally difficult to provide, as building an integrated development environment into the dashboard would have been a huge task, but ultimately it did miss the mark on the original user stories.

5. Management

5.1. Teamwork

From the start of this project, our strategy has had a large focus on both teamwork and accountability. Alongside the weekly engagement clinics in the first half of term, we also organised a weekly team meeting every Friday, and in both of these sessions we would discuss the work each individual had achieved, what was next on our list of priorities, and to end the meetings, we would each make pledges for what we would have completed in time for the next meeting.

This model encouraged everyone to stay up to date, as many tasks would have to be completed in time for the next meeting for another person to begin their next task, et cetera. Also, having such regular collaborations allowed lots of feedback and guidance from one another, meaning changes to workloads and plans were much easier and more flexible, rather than being at the detriment of weeks' worth of work.

While each individual had their own responsibilities, tasks and workload, teamwork was always encouraged, with many examples of two or more people collaborating on a particular task ahead of a meeting, either to reduce the overall time taken, to balance workloads more fairly, or to join together two distinct parts of the project more easily. For example, building the simulation into the dashboard was a joint task, as was connecting the game, database and dashboard so they could all communicate information.

The team roles (outlined below) were also flexible, with workloads being shifted to team members with more available time to ensure project completion and better time management. A key example of this was the sharing of documentation, with several team members taking on these tasks to ensure timely completion ahead of the deadline.

5.2. Contributions and Roles

5.2.1. Sprint 1

During our first team meeting, we discussed the allocation of roles, based on each team member's proficiencies and personal preferences. These ended up being very flexible, with many team members taking initiative and choosing to take on other tasks away from their designated area when they had time, in order to ensure the project's timely completion and ultimate success within this sprint. The officially assigned roles for this sprint were:

Jake Phillips: Project Owner, Data Lead

- Key contributions: created database, Docker containers, READMEs, deployment guide; provided server, coordinated data transfer between system components, helped make presentation

Oliver Robson: Scrum Master, Documentation

- Key contributions: meeting minutes, presentation, presentation scripts, simulation

Doruk Aksu: Tech Lead

- Key contributions: dashboard creation, core functionality, graphs, parameter data management, software data inventory

Dylan Davies: Software Dev

- Key contributions: made the entire game: menus, levels, towers, enemies and gameplay loop

Bela Rothfuss Moore: Quality Assurance & Testing

- Key contributions: all testing throughout the project: unit, integration, automatic, manual end-to-end tests for game, database and dashboard; assessing success criteria; UML for dataflow

Thomas Wilkin-Jones: Game Dev

- Key contributions: project report, game assets, company logo, game lo-fi prototyping, game UML diagrams

Zak Salmon: Dashboard Dev

- Key contributions: dashboard styling and CSS integration, dataflow management for dashboard, lo-fi prototyping for dashboard

As the contributions show, there was a lot of chance for teamwork and collaboration, with some roles being intrinsically linked: Doruk and Zak worked together throughout the sprint to produce the dashboard; Dylan and Thomas collaborated on some aspects of the game; Jake worked with Dylan and Doruk to connect the database to all other parts of the game; and Thomas and Doruk took on documentation work from Oliver. Throughout the project, our team members showed great teamworking skills and willingness to cooperate.

5.2.2. Sprint 2

In the second sprint, we all decided to continue in the same roles as we had allocated in Sprint 1 but with a couple of alterations: as Thomas had taken over the report, he transitioned to Game Assets and Documentation, while Oliver moved into Simulation Developer and Documentation as the simulation had become a huge part of the dashboard's implementation. As such, the roles and contributions new for Sprint 2 were as follows:

Jake Phillips: Project Owner, Data Lead

- Key contributions: database management, password hashing and authentication, handover pack

Oliver Robson: Scrum Master, Sim Dev, Documentation

- Key contributions: updated simulation, presentation slides & scripts, meeting minutes

Doruk Aksu: Tech Lead

- Key contributions: database seeding, user account management, dashboard graphs, anomalies

Dylan Davies: Game Dev

- Key contributions: new game levels, tower upgrades, sound effects, animations, boss enemies, all other game development

Bela Rothfuss Moore: QA & Testing

- Key contributions: weekly manual tests, updated automated tests and end-to-end scenario

Thomas Wilkin-Jones: Game Assets, Documentation

- Key contributions: reinforced enemy assets, boss enemy assets, project report

Zak Salmon: Dashboard Dev

- Key contributions: dashboard functionality, CSS, JavaScript, dashboard styling

6. Risks and Discussion

6.1. Strategy

Our project strategy was to execute lots of smaller, more manageable tasks in short timespans throughout the sprint. These typically followed the nature of our team meeting schedule, with our engagement clinics on a Tuesday and independent meetings on a Friday. At the end of every meeting, every team member would decide what they could feasibly achieve in the next few days ahead of the next meeting, and would be expected to present their completed work next time. Key targets and necessary next steps were always discussed, prompting the choices of subsequent task pledges to reach common goals in time for the next stages to begin.

6.1.1. Pros

The short turnarounds before meetings encouraged the selection of smaller, more realistic and manageable task pledges, as the expectation to have something to show and to be able to tick off what you promised led to better internal reflection of what would be feasible, meaning the whole project's pace flowed a lot quicker and smoother.

Such frequent meetings also encouraged constant communication throughout the project. Everyone was aware of what had been completed, what the next key goals were, and what everyone else was working on, so estimations of the project's trajectory were easier to form, and it was clearer what the next targets should be, and everyone could contribute one what they thought needed changing, or what should be the next priority, meaning scope creep was minimal.

Furthermore, constant communication made it much easier to see when others' workloads were unbalanced, and encouraged teamwork and load balancing to reach overall project goals faster. Being able to pick up other tasks outside the scope of each person's role made project progress much more linear and effective, and allowed team members to explore other roles, rather than being bound strictly to what their field defined.

6.1.2. Cons

Our strategy was not perfect however, as the rapid turnarounds between meetings may have led to unrealistic expectations. Having two mini-sprints per week in between the two meetings means team members were expected to find time in two different intervals during the week, which was often unrealistic. Due to other commitments, convenience and a variety of other reasons, it may not have been feasible for members to find time to complete the expected project work at two separate points of the week, where otherwise it may have been easier to dedicate much more time during one block per week, and this could have caused significant added stress due to the strict expectations of the task pledges.

Also, expectations at the start of the project may have been unrealistic. As this project was entirely new, many of the early task pledges may have been overly ambitious, as members did not yet know the full project scope, or the difficulty of what they were taking on, so progress was far from linear.

Our choices of programming languages may also have affected our strategy and efficiency, with higher expectations than were realistic often causing stress, due to the mix of experience and confidence using these tools. The choice to use Godot meant that a significant amount of time had to be invested into learning the language for team members associated with this, as well as the syntax of CSS and all of the new APIs used in the database and dashboard. Since so much of the software and languages we used was unfamiliar or not taught to the team, lots of independent time had to be invested into learning, and so expectations from those with more experience may have been overbearing on the team members lacking that.

6.2. Project Failure Risk

6.2.1. Sprint 1 Risks

Ultimately, we set out the requirements for this sprint early on, and defined our goals and targets to ensure that we would cover all required bases. We frequently revisited documentation and specifications to ensure we had covered everything we needed, outline what needed more work, and keep track of what needed to be completed next. Failure to cover any of these core requirements would have been a failure for the project as a whole.

One key risk was not having the game built to the required standard. It was ambitious to build a fully functioning game in just five weeks, and having all of the necessary features and functionality was a must, but a significant ask. Ensuring all of the towers worked as intended, enemies dealt damage correctly, the player could win and lose on both of the required two levels, recording at least six telemetry event types and having an adjustable parameter was a lot to do, and as such was a significantly possible point of failure for the project.

The dashboard was the other critical component in our architecture. There were a number of defined criteria for its functionality and different views, including all of the necessary graphs, data analysis opportunities, and the parameter changing functionality, so if anything was missed here, it could be a big risk to the project as a whole.

Assuming both the game and dashboard worked correctly, the most present risk was being unable to link the two together via the database. With so many different languages, apps and APIs used across all of the different parts of the architecture, ensuring smooth communication and data transfer would be challenging, but absolutely critical to the project's success. If the dashboard couldn't read the telemetry events from the game, it would not be able to generate any graphs or display any information about telemetry, being completely blank and useless. And if the game couldn’t receive parameter updates from the dashboard, it would be in a constant fixed state, with no changes or external manipulation possible, meaning difficulty adjustment and balancing would be non-existent, failing to solve the entire problem we had been given.

Our final key risk that we identified was not having all of the required documentation fully up-to-date. Core documents like the project report, ethics and legal considerations and licensing decisions would be critical to evidencing our professionalism, and showing that we had remained fully compliant with all licenses and litigation throughout.

6.2.2. Sprint 2 Risks

Our biggest risk identified in the second sprint was full GDPR compliance. As we needed to have user accounts implemented by the end of the sprint, it was important for us to be able to keep these accounts secure, and make sure that the user data was stored in a manner which would guarantee compliance. Exploring our options, and implementing a robust and safe solution was the biggest task throughout the sprint, as failure here would mean we would either fall short of the requirements, with no user accounts but full GDPR compliance, or have a product that could not be deployed without illegal data collection and storing. Clearly neither of these were desirable scenarios, hence this was our biggest risk in Sprint 2.

6.3. Project Failure Prevention

6.3.1. Sprint 1 Risk Mitigation

While Dylan controlled the game development independently, as he had the most experience in Godot, he was fully transparent in all meetings about where he was in development, what his next targets were, and how long he estimated each part would take to develop. To minimise failure risk, some nice-to-have features were cut from the scope for this sprint, meaning more focus could be attributed to the critical components outlined in the specification and requirements. In the end, cutting some of these features meant that a fully functional game was presented long before the end of the sprint, allowing more time for refinements and Sprint 2 considerations.

The dashboard, though obviously a huge part of the project as a whole, was not much of a risk to failure in actual practice. Consistent collaboration between Zak and Doruk meant that the entire dashboard was fully developed and functional with lots of time to spare, with only the final few integrations like the simulation requiring a bit more work to ensure all of the data was being displayed and transferred as expected. Lots of time could therefore be afforded to visual alterations and refinements to make it as visually pleasing as possible.

Fully accurate and working dataflow was the biggest risk we encountered. With so many different data sources and transfers, and many endpoints in APIs, databases, charts, graphs, and the backend of the game, it was challenging to make sense of the different languages and syntaxes between these different parts of the architecture. Ultimately this issue was resolved by the support of Doruk and Jake as Tech and Data Leads respectively, with Jake helping Dylan to link the game to the database, so the game would post all of its telemetry events, and receive parameter changes, and Doruk managing data across the dashboard to ensure the graphs and tables updated with all of the correct information.

Towards the deadline for this first sprint, documentation was lagging behind our targets due to other commitments and illness which had hindered progress. With other project areas nearing completion, other team members took over key documents to ensure they would all be completed in time, with Thomas writing the Project Report, and Doruk producing the Software Data Inventory.

6.3.2. Sprint 2 Risk Mitigation

In order to remain fully GDPR compliant, we had to ensure a number of things. First, we needed to be able to create user accounts, and separate player and developer accounts in terms of access to the dashboard: only developers should be able to view the telemetry dashboard. To do this, we created a new database schema to store the user account details, then added new signup and login pages to both the game and the dashboard.

Next, we had to make the user accounts secure by protecting the passwords. Jake was able to research open-source hashing algorithms, simultaneously ensuring we could remain on an open source license for this project, finding and implementing the Argon2 algorithm, so that user passwords cannot be decrypted even in the outcome of a database breach.

In addition to this, we needed to make sure that only the developers could access the dashboard, to minimise the threat to the game through malicious targeting of parameters, harming the gameplay and user experience, and to make sure there were not too many conflicts across the dashboard balancing tools. To do this, we had separate account flags to highlight developer/admin accounts, and added the admin requests tab to the dashboard. Then, when an unverified (game only) account attempted to log in to the dashboard, they were prompted to request admin access. This was then shown in the admin requests page for any existing signed in developers or admins, allowing them to accept or deny the request, making the accounts safe.

Finally, since we were now using user accounts throughout the game, we had to change our data collection to make sure it was fully GDPR compliant. As such, we added a disclaimer to the game's login screen to tell the user about how we collect data for the purpose of game balancing tools in the dashboard, while also instructing them on how to delete this data or opt out of data collection. These two options were then implemented in the game's settings page, allowing the player to delete any data associated with their username from the database at the click of a button, and giving them a toggle for opting in or out of future data collection.

All of these user account features were developed safely and proactively to ensure that our finished product remains legal and user friendly. Hence we successfully navigated the biggest threat to the success of this project.

7. Final Evaluation

7.1. Success

Overall, we felt that we succeeded in this project. As assessed in 4.3. Evaluation Results, we hit all of our goals, and almost all of the surveyed end users found the project useful, easy to use, and offered the features they had desired. So, by analysis, this project was a success.

More than that, we all felt that we had enjoyed this project. We worked as a cohesive unit, cooperated, had regular meetings to discuss progress, shared the workload, changed things up, made decisions together, laughed through it, and produced a product we are proud to hand over. In this sense therefore, this project was a resounding success.

7.2. Trade-Offs

We did not suffer many trade-offs throughout the project, as we split all of the tasks and requirements across the whole team, with each person having unique specified leaderships and proficiencies to ensure that every thing that needed doing had an obvious first person to reach out to. We did not often have to allocate time from development of one feature to support another as a positive consequence of this choice of team roles and cooperation, and the whole product ran well on the tools we had meaning we did not have to make sacrifices in the name or performance or memory efficiency, et cetera. Hence we did not experience any trade-offs in the course of development.

7.3. Scope Drift

As discussed in section 1.4. Outside the Scope, there were a handful of features that we had to leave behind in the interest of time and project security, all of which would have been nice to have as additional features and functionality, but were not perhaps necessary according to our user stories and goals. However, as highlighted in 4.3. Evaluation Results, there had been some element of scope drift from our user stories, as we had neglected the user request of having a framework on which to build new levels and implement new features. While this is something the developer could feasibly do, it is not something we natively facilitated or supported or indeed encouraged; hence we had fallen victim to scope drift.

7.4. Consistency and Teamwork

All members of our team collaborated and worked hard to meet the deadlines and requirements set out for this project. Attendance at meetings was consistently high, with all members being transparent about any other commitments preventing attendance in advance, meaning things could be communicated in other ways to ensure everything stayed on track, and new task pledges could be allocated at each meeting. Everyone worked flexibly, with some meetings taking place online for convenience, while most took place in person, on campus. Whenever things were taking longer than expected, or where tasks branched multiple aspects of development and management, members were happy to collaborate to achieve the goals, and reach completion faster. The frequent meetings and checking of task pledges, and updating of the scrum board, ensured that all members were consistent in their work and contributions throughout the project's duration. As such, consistency and teamwork were prevalent throughout our project.

7.5. Next Steps

Should we ever revisit this project, 1.4. Outside the Scope outlines the features we would like to implement next, with more time and resources. While we have a finished product, with nothing fundamental missing, there are elements of styling and graphics which could also be polished, but this is not necessary for our final prototype.

Now that we are finished with the second sprint, however, we are handing over the project to the end users: game players, and game developers. The game has full how-to-play instructions, ensuring new players know what to do in the main gameplay loop, and we assessed how usable the game is and how easy it is to

understand, as mentioned in 4.2. Evaluation Approach. In terms of passing the game over to developers, we have created a handover pack to ensure the project can be run easily with no further input from us. The handover index outlines all parts of the system, and how to deploy and use them all. We also provided a maintenance and troubleshooting guide, so that in the event of any glitches or bugs during normal use, the system can be reset and returned to normal running conditions, and users can continue with what they were attempting to achieve. The game itself can be swapped out for any game built to the same standards in terms of data parsing between itself and the dashboard, ensuring that it adheres to the same schema as the database, so they can run in unison. The database, game and dashboard can also be expanded, with instructions on how to add new telemetry types and parameters to the system, so it can grow beyond the scope of this project.

8. Appendices

8.1. Setup Instructions

Please refer to the deployment_and_operations.pdf file in the 2_handover_pack/ directory.

8.2. API Documents, Endpoints and Schema

Please refer to the README.md file in the database directory in the GitHub repository.

8.3. Test Plan and Test Coverage

Please refer to the testing_evidence.pdf file in the 5_technical/ directory.

9. Report Credits

All parts of this report written by Thomas Wilkin-Jones, except:

- 3.4. Data Analysis, 3.5. Analysis Methods: Doruk Aksu
- 4.1. Tests: Bela Rothfuss Moore
- 8. Appendices: Jake Phillips

Sections 1, 2, 3-3.3, 4.2-4.3, 5, 6, 7, 9: Thomas Wilkin-Jones